The Preprocessor

CSC 230 : C and Software Tools

NC State Department of Computer Science

Topics for Today

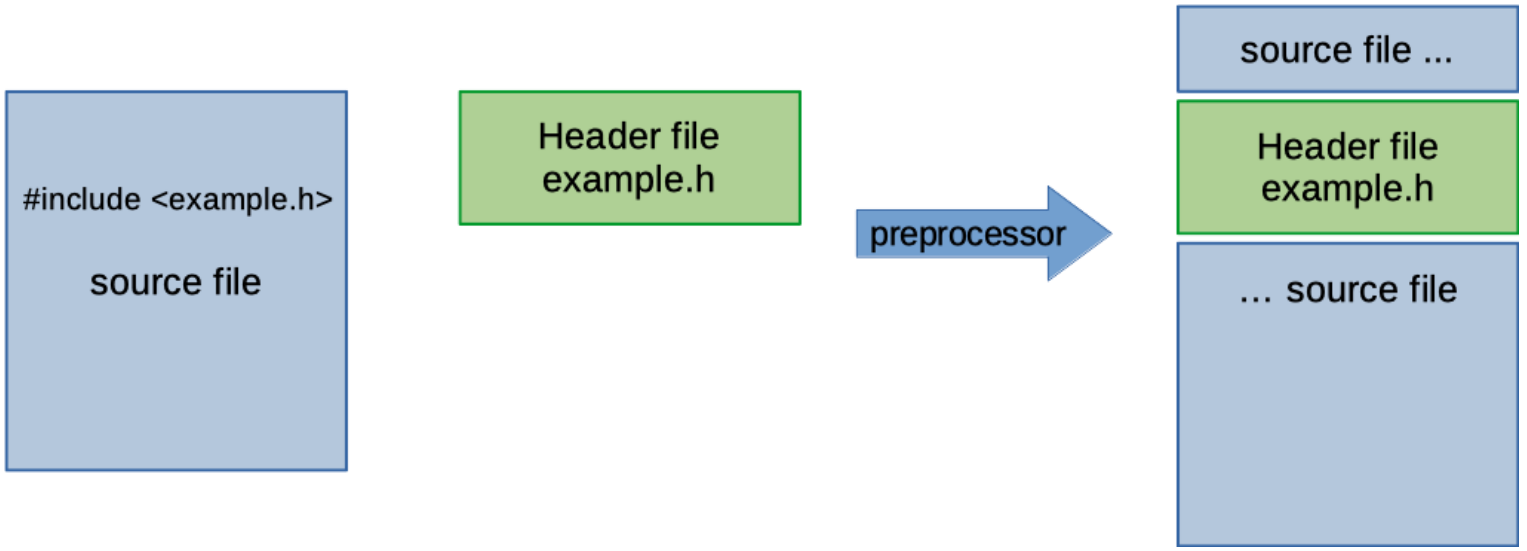
- The preprocessor
- Macros
- Macros with parameters
- Special parameter expansion
- Conditional compilation
- Include guards

Meet the Preprocessor

- The preprocessors : a dumb-ish text manipulation stage before compilation.
- What's it good for?
  - Including fragments of source code (e.g., headers)
  - Defining constants (via macros)
  - Substituting for inline functions (via macros)
  - Conditional compilation

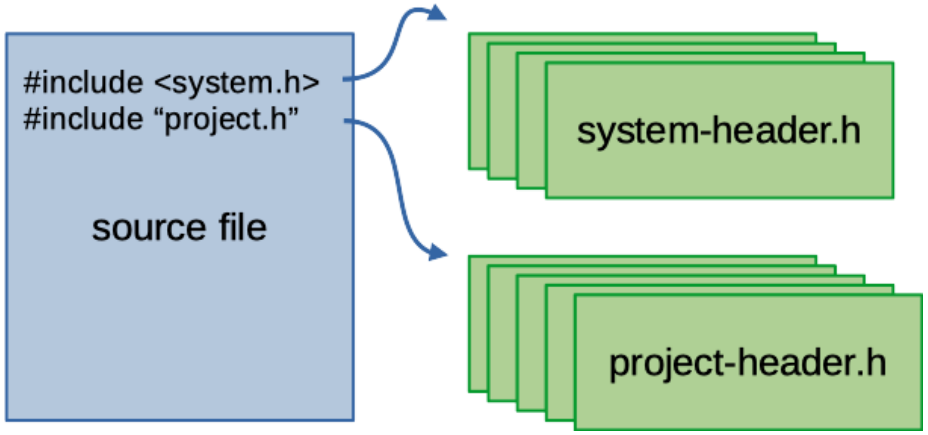
Include Directive

- You know the include directive.
- It copies code from elsewhere
  - Replaces the #include with contents of another file.



Header Types

- Include offers two notations for header names
  - Angle brackets : for including a standard system header.
  - Double quotes : for including files that are part of the current project
- Each of these has a different search path.



- Double quotes includes the current directory in the search path.
- The -I gcc option lets you add more directories.

Preprocessor Macros

- The preprocessor is the first step in compiling your code
  - It does basic text-based operations on your source code
- You know you can use this for named constants
  - During preprocessing, occurrences of this token get replaced by this definition.



  - This is called a *preprocessor macro*
  - This replacement is called *macro expansion*

Macro Expansion

- The preprocessor is a little smart, it looks for whole identifier matching the macro name.

```
#define SIZE 1024
```

- It will substitute in situations like these:

```
int i = SIZE ;
```

```
f( str, (SIZE+1)*2);
```

- But not a situation like this:

```
int list[MAXSIZE];
```

Preprocessor Macros

- Preprocessor macros can expand to multiple tokens.
- This can be useful.

```
#define FAIL exit( 1 )
```

- It can also be a source of some common mistakes:

```
#define SIZE 256;

int list[ SIZE ];
```



```
int list[ 256; ];
```

The preprocessor is dumb.
You have to provide the smarts.

Looking at Macro Expansion

- It can be hard to detect these kinds of errors.

```
#define SIZE 256;
```

  - We don't get to see the intermediate code generated by the preprocessor.
  - But, we can … if we know how to ask.
- The -E option to gcc will send preprocessed output to standard out.

```
gcc -E myProgram.c
```

Get ready for a lot of output.

Preprocessor Directives

- Macro definitions are one example of a *preprocessor directive*
  - Starting with a #
  - Continuing to the end-of-line, but you can escape the newline with \

```
#define LONG_MACRO 1 + 2 + 3 + 4 + 5 + 6 + 7 \
+ 8 + 9 + 10
```

  - Handled by the preprocessor
  - Removed before the compiler gets the code.

Macros within Macros

- Macro definitions can contain other macros

```
#define PI 3.14
#define TWO_PI 2 * PI
```

```
… TWO_PI …
```



```
… 2 * PI …
```



```
… 2 * 3.14 …
```

This also works.

```
#define TWO_PI 2 * PI
#define PI 3.14
```

```
… TWO_PI …
```



```
… 2 * PI …
```



```
… 2 * 3.14 …
```

Macro Parameters

- Macros can take parameters

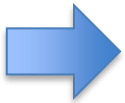
```
#define times2( x ) x * 2
```

Soon, we'll see that this is a bad idea.

Beware. No space here.

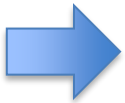
- When the macro expands, the parameter gets copied into the definition

```
int j = times2( i );
```



```
int y = i * 2;
```

```
a[i] = times2(a[k]) + 1;
```



```
a[i] = a[k] * 2 + 1;
```

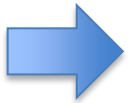
- This is a mechanism for call-by-name

Macros as Functions

- Macros can substitute for inline-functions.

```
#define MAX( x, y ) x > y ? x : y
```

```
int i = …, j = …;
int k = MAX( i, j );
```

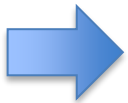


```
int i = …, j = …;
int k = i > j ? i : j;
```

No function-call overhead.

- And, we can use them to write code that's independent of type.

```
double a = …, b = …;
double c = MAX( a, b );
```



```
double a = …, b = …;
double c = a > b ? a : b;
```

Faster Character I/O

- The standard library includes some macros.
- We have functions for character I/O
  - So you pay a little function call overhead with every call.

```
int fgetc(FILE *fp);
int fputc(int ch, FILE *fp);
```

I'm a function.

- We also have macros that do the same job … only faster.
  - These expand to code right in place of the call.

```
int getc(FILE *stream);
int putc(int c, FILE *stream);
```

I may be a macro.

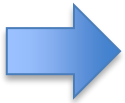
Planning for Expansion

- So far, all of our macros have been poor examples … like this one:

```
#define times2( x ) x * 2
```

- What happens if you use it like this?

```
int j = times2( i + 1 );
```



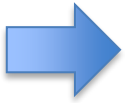
```
int j = i + 1 * 2;
```

Oops, I think we wanted (i + 1) * 2

- We can use parentheses to suppress this problem.

```
#define times2( x ) (x) * 2
```

```
int j = times2( i + 1 );
```



```
int j = (i + 1) * 2;
```

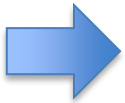
Planning for Expansion

- We can get similar problems with precedence of surrounding expression syntax.

```
#define times2( x ) (x) << 1
```

- What happens if you use it like this?

```
int j = times2( i ) + 1;
```



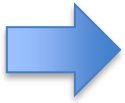
```
int j = (i) << 1 + 1;
```

Oops, I think we wanted (i << 1) + 1

- Parentheses around the whole thing will fix this.

```
#define times2( x ) ((x) << 1)
```

```
int j = times2( i ) + 1;
```



```
int j = ((i) << 1 ) + 1;
```

Protecting the Macro Definition

- Macros that work like functions
  - They need protection with parentheses.

```
#define MAX( x, y ) ( (x) > (y) ? (x) : (y) )
```

- Macros that work like statements
  - They usually need protection with curly brackets

```
#define PRINT2( x, y ) { \
    printf( "%d\n", x ); \
    printf( "%d\n", y ); \
}
```

- Some macros don't fall into either category.

Macros vs Functions

- Macros can work as a substitute for function calls
- … but, they can be more difficult to use correctly.
  - They can expand to more code than you would get with a function call
  - They can cause problems when their parameters have side effects

Macro Expansion of Macro Expansion of …

- Consider this handy little macro

```
#define MAX( x, y ) ( (x) > (y) ? (x) : (y) )
```

- What if you need the maximum of 3 or more items?
  - We can nest uses of the macro
  - For 3 parameters

```
MAX( a, MAX( b, c) )
```

  - Or for 4:

```
MAX( MAX( a, b ), MAX( c, d ) )
```

Macro Expansion of Macro Expansion of …

```
#define MAX( x, y ) ( (x) > (y) ? (x) : (y) )
```

- Nested macro calls could expand to a lot of code.

```
MAX( MAX( a, b ), MAX( c, d) )
```



```
( (MAX( a, b )) > (MAX( b, c )) ? (MAX( a, b )) : (MAX( c, d )) )
```



```
( (( (a) > (b) ? (a) : (b) )) >
  (( (c) > (d) ? (c) : (d) )) ?
  (( (a) > (b) ? (a) : (b) )) :
  (( (c) > (d) ? (c) : (d) )) )
```

Hidden Overhead

- Generated code could be inefficient

```
MAX( MAX( sqrt(a), sqrt(b) ), MAX( sqrt(c), sqrt(d) ) )
```



```
( (( (sqrt(a)) > (sqrt(b)) ? (sqrt(a)) : (sqrt(b)) )) >
  (( (sqrt(c)) > (sqrt(d)) ? (sqrt(c)) : (sqrt(d)) )) ?
  (( (sqrt(a)) > (sqrt(b)) ? (sqrt(a)) : (sqrt(b)) )) :
  (( (sqrt(c)) > (sqrt(d)) ? (sqrt(c)) : (sqrt(d)) )) )
```

This will call sqrt() nine times.

Extra Side-Effects

- Code gets duplicated during macro expansion
- This can cause problems when the code has side effects.

```
z = MAX( x++, y++ );
```



```
z = ( (x++) > (y++) ? (x++) : (y++) );
```

One of these will get incremented twice.

Macros and Side Effects

- It's the programmer's job to consider possible errors like this.

```
z = MAX( x, y );
x++, y++;
```

That's better.

- This is why the documentation warns you about calls that may be implemented as a macro
- We saw this with the character I/O functions.
  - fgetc() vs getc()
  - fputc() vs putc()
  - …

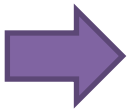
These may be macros.

Macros Can See The Parameters

- Macros use the **text** of their parameters
  - This is an example of *call-by-name*
- With this, we can do things a function can't

```
#define SWAP(a, b, type)  \
{                         \
    type temp = a;        \
    a = b;                \
    b = temp;             \
}
…
    int x = 5;
    int y = 7;
    …
    SWAP(x, y, int);
```

Would be nice to avoid this.



A copy of the parameter text.

```
    int x = 5;
    int y = 7;
    {
      int temp = x;
      x = y;
      y = temp;
    }
```

Copying Variables as Memory

- We can simplify this macro
  - A variable is stored in a little block of memory
  - We can let the macro access that memory.

```
#define SWAP(a, b)                          \
{                                           \
  char temp[sizeof(a)];                     \
  memcpy( temp, &a, sizeof(a) );  \
  memcpy( &a, &b, sizeof(a) );    \
  memcpy( &b, temp, sizeof(a) ); \
}
…
   int x = 5;
   int y = 7;
   …
   SWAP(x, y);
```

Size of temp depends on a.

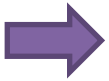
This will copy any contents of memory.

Accessing Parameter Text

- Normally, the parameter is expanded as code.
- You can get it as a quoted string.
  - Just put a **#** in front.

```
#define hello(x)          \
   printf("hello %s",     \
          #x)
...

  hello(world);
  hello(kitty);
  hello(darkness my old friend);
```



```
printf("hello %s",
       "world");
printf("hello %s",
       "kitty");
printf("hello %s",
       "darkness my old friend");
```

Help with Debugging

- Here, call-by-name lets us do things we couldn't with a function.
- We can give more context in our debug output

```
// Macro to print the value of an int, along with
// its name.
#define REPORT( X ) printf( "%s = %d\n", #X, (X) )

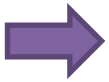
…

  REPORT( i );
  REPORT( total );
```

Concatenating Parameters

- Normally, you get spaces or punctuation around parameter values.
- You may want to build an identifier name using a parameter value.
- **##** soaks up space between neighboring syntax.

```
#define nameGame(x,y)       \
{                           \
  int x ## _limit;          \
  char y ## Text = "abc";   \
  double x ## y = 98.6;     \
  ...
}
…
  nameGame( red, green );
```



```
{
  int red_limit;
  char greenText = "abc";
  double redgreen = 98.6;
  ...
}
```

Built-In Macros

- Built-In macros can give more details about the source code.
  - The __LINE__ macro gives the current line number
  - The __FILE__ macros gives the current filename
  - (that's two underscores at the start and end)

```
#define REPORT( X )                                  \
   printf( "%s(%d) : %s = %d\n",                       \
           __FILE__, __LINE__, #X, (X) )

...

  REPORT( i );
  REPORT( total );
```

Note, these get expanded after the REPORT macro is expanded.

Built-In Macros

- Built-in macros can give you information about when a program was compiled.
  - The __DATE__ macro expands to the current date
  - The __TIME__ macro expands to the current time
  - (also two underscores at the start and end)

```
void about()
{
   printf( "My program\n" );
   printf( "Compiled: %s %s\n", __DATE__, __TIME__ );
}
```

These expand to strings at compile time.

Conditional Compilation

- We can tell the preprocessor to keep or discard blocks of code.
- Common uses:
  - Handle platform dependencies
  - Enable or disable debugging code
- Here's what conditional compilation looks like

```
#if...

someCodeYouMayWant();
orMayNotWant();

#endif
```

The preprocessor may discard this code, depending on the directive above.

Conditional Compilation

- We have lots of (related) directives for writing condition compilation
  - Starting a block of conditionally-compiled code

```
#if
```

```
#ifdef
```

```
#ifndef
```

  - Between conditionally-compiled alternatives

```
#else
```

```
#elif
```

  - Ending a block of conditionally-compiled code.

```
#endif
```

Preprocessor Expressions

- We can ask the preprocessor if a macro name is already defined.

```
#if defined(MACRO_NAME)
```

- This is so common, it has short-hand syntax.

```
#ifdef MACRO_NAME
```

```
#ifndef MACRO_NAME
```

- We can ask about values of macros

```
#if MACRO_X == 25
```

- We can even build compound conditionals

```
#if MACRO_X > 25 && MACRO_Y < 30
```

Conditional Compilation: Example

```
#if defined(LINUX)
  #define HDR "linux.h"
#elif defined(WIN32)
  #define HDR "windows.h"
#else
  #define HDR "default.h"
#endif

#include HDR
```

- And when compiling this program, can choose a system using the **-D** option to **gcc**

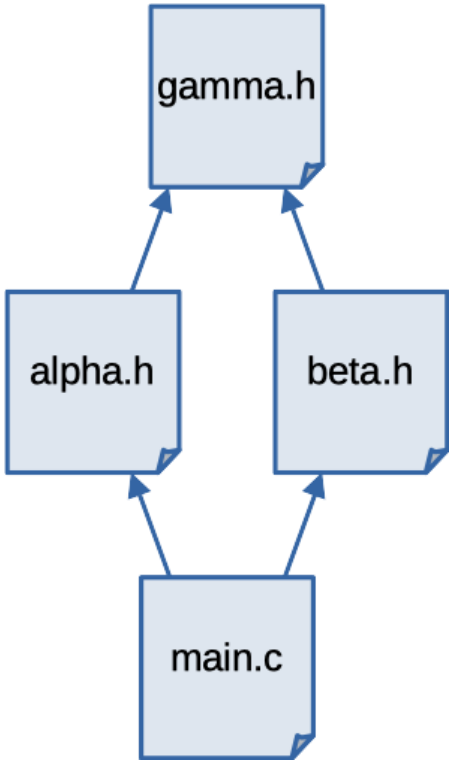


```
gcc -DWIN32 myprog.c …
```

```
#include "windows.h"
```

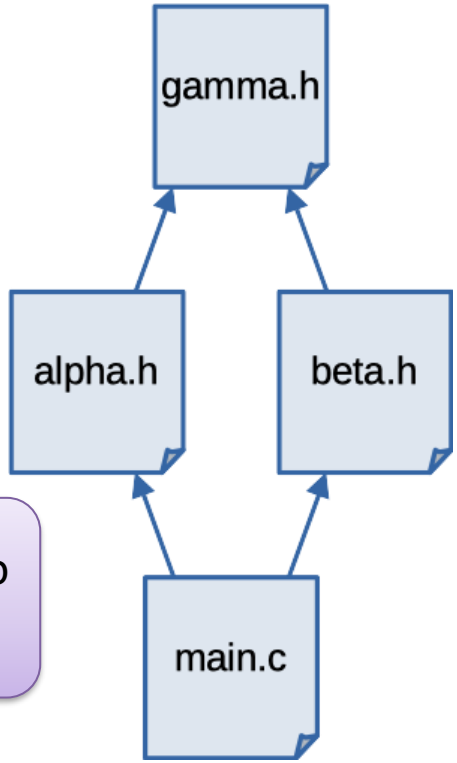
Include Guards

- It's possible to include the same header more than once … even without trying to.
- Is this bad?
  - Well, that's extra work for the compiler
  - And, if the header contains definitions, we're in trouble.
  - You can declare something as many times as you like, but you better only define it once.
- How can we fix this?
  - You guessed it, with the preprocessor.



Include Guards

- An include guard is a preprocessor trick that.
  - Keeps the contents of the header the first time it occurs.
  - Discards its contents on subsequent occurrences.




```
#ifndef GAMMA_H
#define GAMMA_H

… code
… inside
… gamma.h

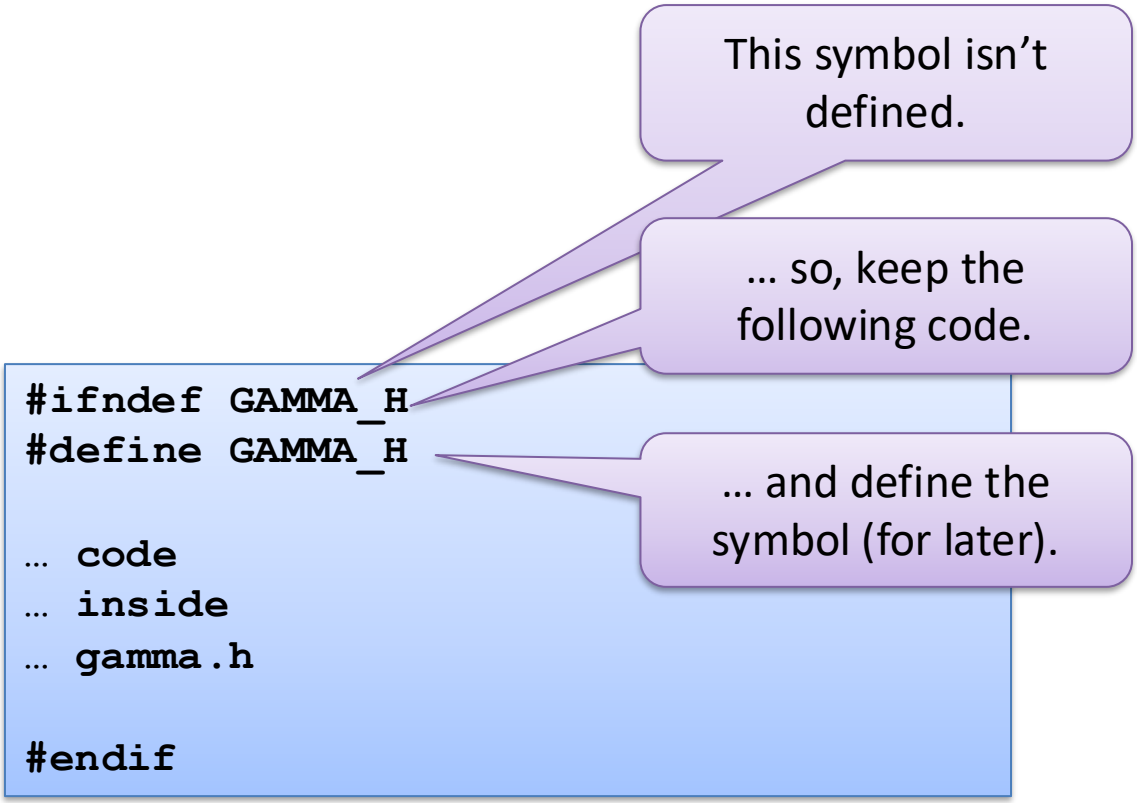
#endif
```

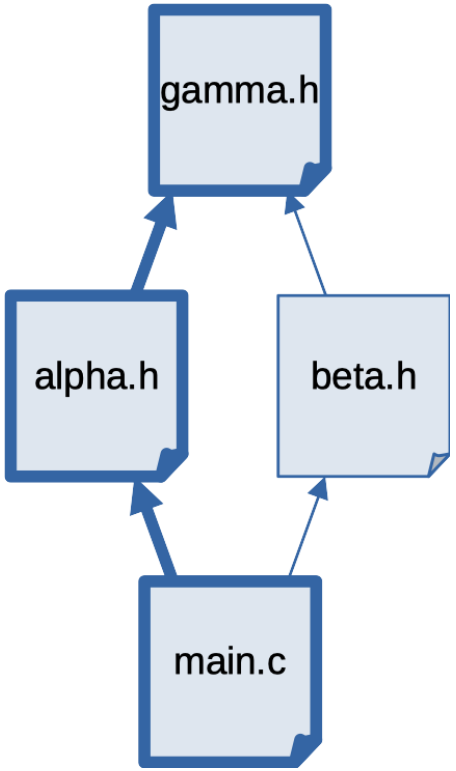
Some name unique to this header.

Used as a flag it indicate the header has already been included.

Include Guards

- The first time the header is included.





Include Guards

- If it's included from later via some other header …

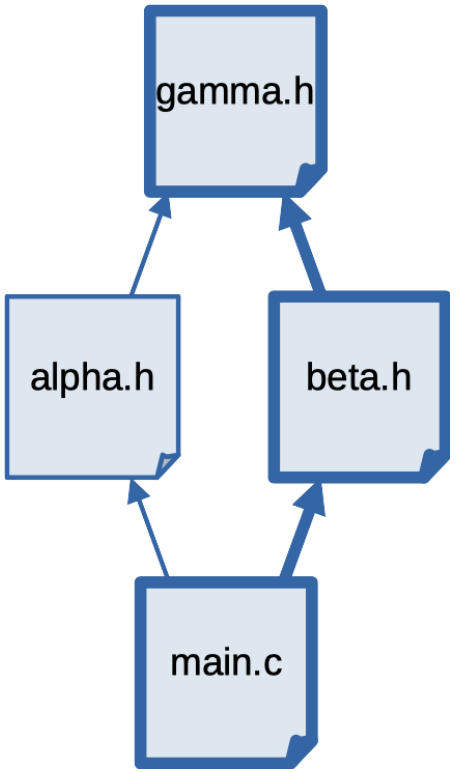
Now, this symbol is defined.

… so, discard the following code.

```
#ifndef GAMMA_H
#define GAMMA_H

… code
… inside
… gamma.h

#endif
```



Help with Debugging

- Conditional compilation can let us exclude debugging code when we don't need it.

```
#ifdef DEBUG

    …;
    #ifdef DEBUG
        …;
    #endif
    …;

#endif
```

Unlike comments, these are guaranteed to nest.

- Then, we can re-enable it if we need it again.

Conditional Debugging Code

- Conditional compilation lets us choose code to use
- We can choose the definition for a macro
  - Even disabling some of them.

```
#if defined(DEBUG)
#define REPORT( X ) printf( "%s = %d\n", #X, (X) )
#else
#define REPORT( X )
#endif

...

  REPORT( i );
  REPORT( total );
```

Using this definition removes uses of this macro.

Definitions on the Command Line

- On the command line, we can choose preprocessor symbols to define.
- This lets us enable / disable program features without changing source code.

```
gcc -DDEBUG -std=c99 -Wall …
```

Any macro you want to define.

I'm defined to **1**

```
gcc -DNAME=BILL "-DMESSAGE=HELLO WORLD" -std=c99 …
```

I'm set to BILL

With help from the shell, you can include spaces.